

# Diseño de IOT y sistemas embebidos

## Laboratorio 2

---

TouchPad, WiFi y Servidor Web

*Docentes:*

*Nicolas Calarco y Pablo Alonso*

*Estudiantes:*

*Victoria Amor, Nicolas Meyer y Bruno Moreira*

**14 de Mayo de 2026**

## Introducción

En el presente laboratorio se aborda la integración progresiva de distintos periféricos y servicios del sistema embebido con el objetivo de ampliar las capacidades de interacción y conectividad del dispositivo. En particular, se incorpora la utilización de la placa de expansión ESP-LyraP-TouchA v1.1 para la lectura de entradas mediante botones capacitivos, permitiendo la detección de eventos de interacción directa con el usuario.

Asimismo, se introduce la configuración del módulo WiFi en sus dos modos principales de operación: estación (STA), donde el dispositivo se conecta a una red existente, y punto de acceso (AP), en el cual el propio sistema genera una red inalámbrica. Esta funcionalidad resulta fundamental para habilitar la comunicación con otros dispositivos y constituye la base para el desarrollo de aplicaciones IoT.

Finalmente, se implementa un servidor web en la placa, al cual se accede conectando una computadora o teléfono a la red WiFi generada por el sistema. Desde la página web es posible consultar el estado del LED RGB y modificarlo de forma remota. A su vez, el LED también puede controlarse localmente mediante los botones capacitivos del TouchPad, integrando así el control físico y el control desde la interfaz web en una misma aplicación.

### 1. Botones capacitivos (TouchPad)

#### 1.1 Configuración del proyecto base

En primer lugar, se creó un nuevo proyecto partiendo nuevamente de una plantilla base (`sample project`) adaptada al microcontrolador ESP32-S2. A partir de esto, se reutilizaron las librerías desarrolladas en el laboratorio uno para el control del LED RGB y la generación de retardos temporales. En particular, la librería `rgb_led` permite el manejo del LED utilizando internamente la librería `led_strip`, la cual se encarga de la generación y transmisión de los datos hacia el LED. Por otra parte, la librería `delay` se encarga del uso de retardos en milisegundos y segundos.

Finalmente, se verificó el correcto funcionamiento del entorno compilando el proyecto y ejecutando un programa mínimo que permite encender y apagar el LED RGB. Esto asegura que tanto la estructura del proyecto como las librerías integradas se encuentran correctamente configuradas antes de avanzar con el desarrollo de nuevas funcionalidades que las utilicen.

En la Figura 1 se presenta el código utilizado para el control del LED RGB. A diferencia del laboratorio anterior, la librería `led_strip` está ubicada dentro de la carpeta del componente `rgb_led`, por lo que fue necesario modificar el archivo `CMakeLists.txt` de dicho componente. En particular, se agregó explícitamente el archivo de implementación `led_strip/src/led_strip_rmt_ws2812.c` y la carpeta donde se encuentran los archivos `.h`, los cuales contienen las declaraciones de las funciones (`led_strip/include`), de modo que ESP-IDF pudiera compilar correctamente el driver y encontrar el archivo `led_strip.h` (Figura 2).

```

1  #include <stdio.h>
2  #include "rgb_led.h"
3  #include "delay.h"
4
5  #define BLINK_PERIOD_MS 500
6
7  void app_main(void)
8  {
9      rgb_led_init();
10
11      while (1) {
12          printf("LED RGB encendido (amarillo)\n");
13          rgb_led_set_color(255, 255, 0);
14          delay_ms(BLINK_PERIOD_MS);
15
16          printf("LED RGB apagado\n");
17          rgb_led_off();
18          delay_ms(BLINK_PERIOD_MS);
19      }
20 }

```

Figura 1: Código de control del LED RGB.

```

components > rgb_led > CmakeLists.txt
1  idf_component_register(
2      SRCS
3          "rgb_led.c"
4          "led_strip/src/led_strip_rmt_ws2812.c"
5      INCLUDE_DIRS
6          "include"
7          "led_strip/include"
8      PRIV_REQUIRES
9          esp_driver_rmt
10 )

```

Figura 2: Modificación del CMakeLists.txt del componente rgb\_Led.

Asimismo, al igual que en el laboratorio anterior, se desactivó el watchdog, dado que no resulta necesario para el funcionamiento del sistema implementado. Por otra parte, se corrigió una advertencia relacionada con el tamaño de la memoria flash, ya que el sistema detectaba una capacidad de 2 MB cuando en realidad la placa cuenta con 4 MB. Por ello, se modificó la configuración desde el menuconfig, en la ruta Serial flasher config → Flash size, ajustándolo al valor de 4 MB.

A continuación, se realizó la conexión física del módulo ESP-LyraP-TouchA v1.1 a la placa base mediante el cable FPC, el cual vincula directamente ambas placas a través de sus respectivos conectores. Este cable transporta las señales necesarias para el funcionamiento del touchpad, incluyendo alimentación y líneas de comunicación (Espressif Systems, s.f).

Al revisar la documentación y el esquemático de la ESP32-S2-Kaluga-1 v1.2 se observa que es necesario verificar la posición de los interruptores asociados al

TouchPad. En particular, los Touch Switch deben colocarse en posición OFF para que los GPIO1 a GPIO14 queden conectados a las líneas táctiles utilizadas por la placa ESP-LyraP-TouchA. Si estos interruptores se colocan en posición on, esos GPIO quedan disponibles para otros usos y el módulo táctil puede no funcionar correctamente. En particular, el módulo hace uso de las líneas de sensores táctiles del ESP32-S2 (canales ESP\_TOUCH1 a ESP\_TOUCH14, asociados a los pines GPIO1 a GPIO14), las cuales se encuentran enrutadas hacia el conector FPC correspondiente (Espressif Systems, s. f.).

Sin embargo, la documentación indica limitaciones asociadas al uso simultáneo de múltiples módulos, debido al multiplexado de pines. En el caso del touchpad, se identifican conflictos principalmente en los pines I01, I02, I03, I06 e I011. Por ejemplo, al utilizar el touchpad junto con el módulo de cámara (ESP-LyraP-CAM), no es posible usarlos simultáneamente debido al uso compartido de los pines I01, I02 e I03. Asimismo, al combinar el touchpad con el módulo LCD (ESP-LyraP-LCD32), se generan conflictos en I06 e I011, lo que impide el correcto funcionamiento de algunas funciones táctiles si dichos pines son inicializados. En estos casos, la solución consiste en no inicializar los pines en conflicto, aunque esto implica la pérdida de ciertas funcionalidades del touchpad (Espressif Systems, s. f.).

## **1.2 Lectura de botones por polling**

El touchpad del ESP32-S2 permite detectar pulsaciones mediante sensores capacitivos integrados en el microcontrolador. Cada botón táctil consiste en un electrodo, es decir, una superficie metálica presente en la placa. Cuando el usuario acerca o apoya el dedo sobre el sensor, se genera una interacción eléctrica entre la persona y el electrodo, lo que modifica la capacitancia del sistema.

Internamente, el periférico realiza ciclos de carga y descarga sobre este electrodo. El tiempo que tarda este proceso depende de la capacitancia: cuando el sensor no está siendo tocado, la carga y descarga ocurre más rápidamente, mientras que al tocar el electrodo el proceso se vuelve más lento, modificando la pendiente de la señal. Este comportamiento puede visualizarse en la forma de onda generada, la cual presenta una señal periódica en forma de diente de sierra, cuya pendiente está directamente relacionada con la capacitancia del sensor (Figura 3) (Espressif Systems, s. f.).

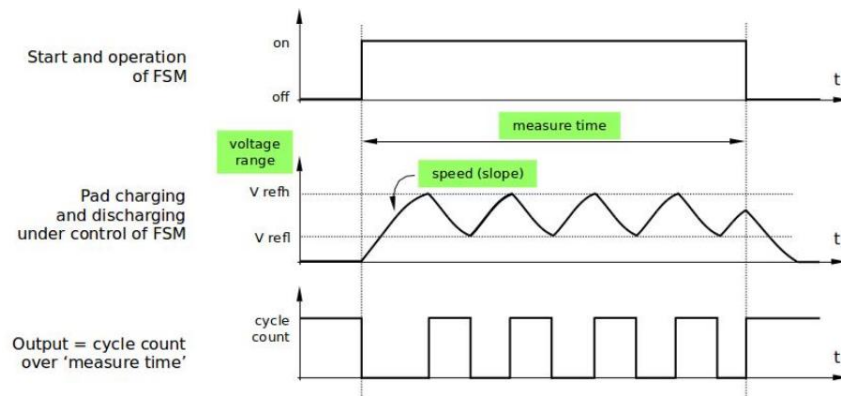


Figura 3: Gráfica de la señal y la cantidad de ciclos medidos en un tiempo dado.

El valor que utiliza el sistema corresponde al número de ciclos de carga y descarga registrados durante un intervalo de tiempo fijo de medición. Es decir, el periférico fija un tiempo de medición y cuenta cuántos ciclos ocurren dentro de dicho intervalo, obteniendo así un valor numérico que representa el estado del sensor. Este proceso es realizado automáticamente por una máquina de estados finitos (FSM) integrada en el hardware, la cual gestiona la inicialización, habilitación y ejecución del proceso de medición sin intervención directa del procesador (Espressif Systems, s. f.). La Figura 4 muestra el diagrama de estados del driver del sensor táctil, en el cual se observa el proceso de inicialización del controlador, la habilitación del periférico y la ejecución de las mediciones mediante el estado de escaneo. El sistema puede operar en modo continuo o por mediciones individuales, retornando al estado de espera una vez completado el escaneo de los canales.

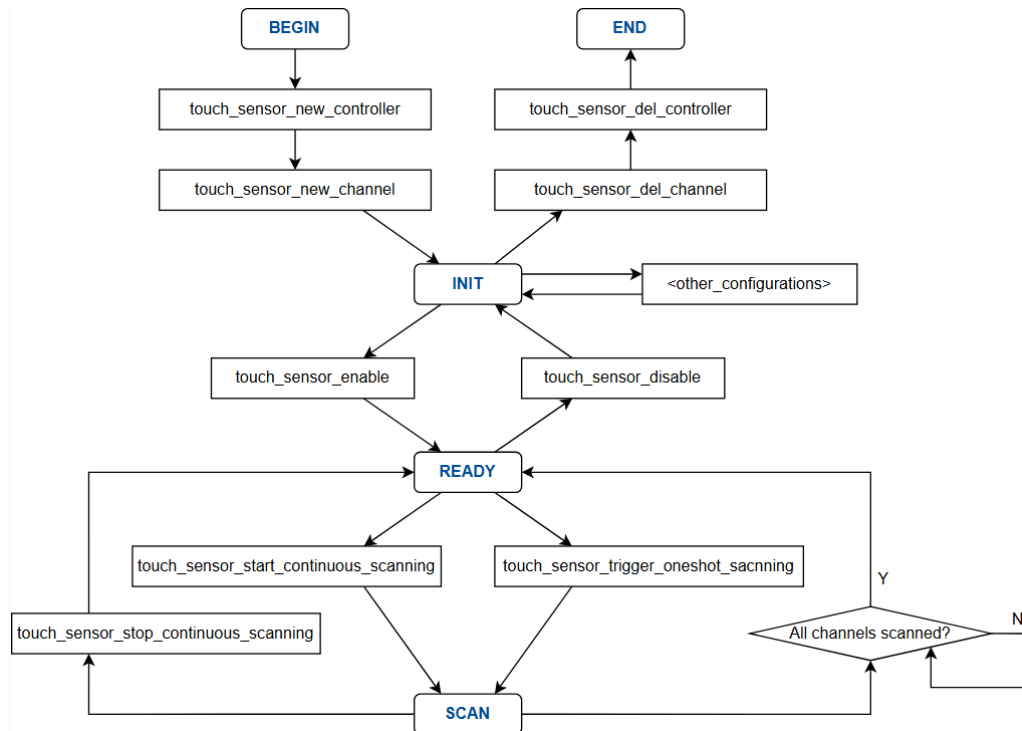


Figura 4: FSM (diagrama de estados del touch).

Para determinar si un botón está presionado, se define un valor de umbral (threshold), que actúa como referencia para comparar las mediciones obtenidas. En el caso del ESP32-S2, una pulsación produce una disminución en la cantidad de ciclos medidos dentro del tiempo fijo, debido a que el aumento de la capacitancia ralentiza el proceso de carga y descarga. Por lo tanto, el estado del botón puede determinarse evaluando si la lectura se encuentra por debajo del umbral establecido. Este valor no es fijo, ya que depende de factores como el diseño del electrodo, el entorno y el ruido eléctrico. Por este motivo, a veces se determina experimentalmente midiendo los valores del sensor en estado libre y presionado, y seleccionando un valor que permita diferenciar ambos estados de forma “confiable” (Espressif Systems, s. f.).

Luego de describir el funcionamiento del touchpad y cómo se utiliza el umbral para distinguir si un botón está presionado o no, se presenta a continuación, en forma de pseudocódigo, una implementación basada en *polling*. En este enfoque, el programa revisa periódicamente el estado de cada botón, compara el valor medido con su umbral y detecta cuándo un botón se presiona o se libera a partir del cambio entre su estado actual y el anterior.

```
Inicializar el TouchPad
Definir array botones[6] con los canales a utilizar
Para cada botón i en botones:
    Configurar canal táctil i
    Medir valor en reposo
    Definir threshold[i] a partir de ese valor
Inicializar estado_anterior[6] en NO_PRESIONADO
Mientras (verdadero):
    Para cada botón i en botones:
        Leer valor actual del sensor → valor
        Si (valor < threshold[i]) entonces estado_actual = PRESIONADO
        Sino estado_actual = NO_PRESIONADO
        Si (estado_actual == PRESIONADO y estado_anterior[i] == NO_PRESIONADO)
        entonces Evento: BOTÓN i PRESIONADO
        Si (estado_actual == NO_PRESIONADO y estado_anterior[i] == PRESIONADO)
        entonces Evento: BOTÓN i LIBERADO
        estado_anterior[i] = estado_actual
    Fin para
    Esperar un poco(delay) //Para evitar leer datos constantemente y así
    reducir el uso de la CPU
Fin mientras
```

Una vez entendido el funcionamiento del touchpad y agregado correctamente el control del LED RGB y los retardos temporales, se procedió a incorporar el soporte para sensores táctiles capacitivos dentro del proyecto. Para esto no se utilizó una librería externa descargada manualmente, sino el sistema de gestión de componentes propio de ESP-IDF.

En particular, se utilizó el comando `idf.py add-dependency "espressif/touch_element"`, el cual permite descargar e integrar automáticamente un componente oficial de Espressif dentro del proyecto. Como resultado de este proceso, ESP-IDF crea la carpeta `managed_components/`, donde se almacena el componente incorporado. Dentro de esta carpeta aparece `espressif__touch_element`, que contiene la implementación desarrollada por Espressif para trabajar con sensores táctiles capacitivos.

Este componente incluye múltiples archivos como `touch_button.c`, `touch_slider.c` y `touch_matrix.c`, los cuales corresponden a distintas formas de interpretar los sensores táctiles, ya sea como botones individuales, controles deslizantes o matrices. En el contexto de este laboratorio, el interés se centra en el uso como botones, es decir, en determinar si un electrodo está siendo tocado o no.

El aporte principal de esta librería es que ya implementa toda la lógica necesaria para interactuar correctamente con el periférico Touch Sensor del ESP32-S2. Este periférico es hardware interno del microcontrolador y se encarga automáticamente de realizar el proceso de medición sobre los electrodos, basado en ciclos de carga y descarga. El resultado de este proceso no es una capacitancia en unidades físicas, sino un valor numérico proporcional al comportamiento del sensor durante un intervalo de tiempo fijo. Por lo tanto, la detección de una pulsación no se basa en medir directamente la capacitancia, sino en observar la variación de este valor numérico entregado por el periférico.

La librería `touch_element` se encarga internamente de configurar los canales táctiles, iniciar las mediciones, obtener estos valores y compararlos contra referencias para determinar eventos como “presionado” o “liberado”. De esta forma, evita tener que implementar desde cero toda la interacción con el hardware. Dentro del componente descargado automáticamente por ESP-IDF, se identificó que en la dirección `managed_components/espressif__touch_element/examples/touch_button` existe un ejemplo oficial provisto por Espressif para el uso de botones táctiles. A partir de este ejemplo se tomó como base la estructura de inicialización y creación de botones.

El primer paso que indica Espressif es inicializar la librería general mediante `touch_element_install()`. Para esto se utiliza una configuración global de tipo `touch_elem_global_config_t`, y la documentación recomienda usar el inicializador por defecto `TOUCH_ELEM_GLOBAL_DEFAULT_CONFIG()`, ya que es adecuado para aplicaciones generales y evita modificar parámetros internos del periférico sin conocer completamente su efecto (Figura 5).

```
// Config general de la libreria touch de espressif
touch_elem_global_config_t global_cfg = TOUCH_ELEM_GLOBAL_DEFAULT_CONFIG();
ESP_ERROR_CHECK(touch_element_install(&global_cfg));
```

*Figura 5: Inicialización de la librería general touch\_element*

Después de inicializar el sistema general, es necesario inicializar el tipo de elemento táctil a utilizar. En este caso, se trabaja con botones capacitivos individuales, por lo que se debe inicializar el módulo de botones mediante `touch_button_install()`. Para ello, se define una configuración global de tipo `touch_button_global_config_t` utilizando el inicializador `TOUCH_BUTTON_GLOBAL_DEFAULT_CONFIG()`, el cual toma valores por defecto según el ejemplo provisto por Espressif (Figura 6).

```
// Config general para botones capacitivos
touch_button_global_config_t button_global_cfg = TOUCH_BUTTON_GLOBAL_DEFAULT_CONFIG();
ESP_ERROR_CHECK(touch_button_install(&button_global_cfg));
```

*Figura 6: Inicialización del módulo de botones táctiles mediante touch\_button\_install()*

Con esto se inicializa el bloque de botones táctiles. Es decir, primero se inicializa el sistema `touch_element` completo y luego se habilita el uso específico de botones.

Luego se crean los botones individuales. La documentación menciona que para crear una instancia nueva de botón se debe usar `touch_button_create()`, seleccionando el canal táctil correspondiente y asignando una sensibilidad. También se aclara que un botón táctil consume un canal del sensor. Por eso, se definió un arreglo con los canales de la placa ESP-LyraP-TouchA, de modo que cada posición del arreglo represente un botón lógico del programa. Esta estructura se tomó del ejemplo visto, donde también se utiliza un arreglo de canales para representar varios botones. Sin embargo, en este caso se adaptó a la cantidad de botones del touchpad según el esquemático, donde se indica qué pines del ESP32-S2 están conectados a cada botón táctil.

Siguiendo ese mismo ejemplo, también se utilizó un arreglo de tipo `touch_button_handle_t` para almacenar los botones creados (Figura 7). Este *handle* no es un número de canal ni un valor físico, sino una referencia interna que devuelve la librería al llamar a `touch_button_create()`. Es decir, cuando se crea un botón, la librería genera una instancia interna y devuelve este identificador, que luego se utiliza para poder referirse a ese botón, por ejemplo, al configurarlo o al manejar sus eventos.



```
static const touch_pad_t touch_channels[TOUCHPAD_BUTTON_COUNT] = {
    TOUCH_PAD_NUM1,    // IO1
    TOUCH_PAD_NUM2,    // IO2
    TOUCH_PAD_NUM3,    // IO3
    TOUCH_PAD_NUM5,    // IO5
    TOUCH_PAD_NUM6,    // IO6
    TOUCH_PAD_NUM11    // IO11
};

static touch_button_handle_t button_handle[TOUCHPAD_BUTTON_COUNT];
```

*Figura 7: Definición de los canales táctiles y del arreglo de handles para los botones.*

Posteriormente, apoyándose nuevamente en la estructura observada en el ejemplo de Espressif, se recorre el arreglo de botones mediante un bucle for. En cada iteración se toma el canal táctil correspondiente y se construye una estructura de configuración de tipo `touch_button_config_t`, donde se especifica el canal (`channel_num`) y la sensibilidad (`channel_sens`), y luego se crea la instancia del botón mediante la función `touch_button_create()` (Figura 8).

De esta manera, cada botón físico de la placa queda asociado a una instancia interna manejada por la librería, cuyo identificador (`handle`) se almacena en el arreglo previamente definido y explicado.

```
for (uint8_t i = 0; i < TOUCHPAD_BUTTON_COUNT; i++) {
    touch_button_config_t button_cfg = {
        .channel_num = touch_channels[i],
        .channel_sens = 0.10f,    // sensibilidad del boton, si no detecta se puede bajar
    };

    ESP_ERROR_CHECK(touch_button_create(&button_cfg, &button_handle[i]));
}
```

*Figura 8: Creación de los botones recorriendo el arreglo de canales y utilizando `touch_button_create()`.*

Después de crear cada botón, es necesario indicar qué eventos se desean detectar para poder reaccionar ante la interacción del usuario. En este caso, interesan los eventos de pulsación (`TOUCH_ELEM_EVENT_ON_PRESS`) y liberación (`TOUCH_ELEM_EVENT_ON_RELEASE`), ya que permiten distinguir cuándo el usuario toca el botón y cuándo deja de hacerlo.

Para lograr esto, se utiliza la función `touch_button_subscribe_event()` (según la estructura del ejemplo), la cual configura cada botón para que la librería genere una especie de notificación cuando ocurra alguno de estos eventos (Figura 9).

```
ESP_ERROR_CHECK(touch_button_subscribe_event(
    button_handle[i],
    TOUCH_ELEM_EVENT_ON_PRESS | TOUCH_ELEM_EVENT_ON_RELEASE,
    (void *)(uintptr_t)i
));
```

*Figura 9: Suscripción a eventos mediante la función touch\_button\_subscribe\_event().*

Luego se define cómo la librería va a avisar de esos eventos al programa principal. La documentación plantea dos formas: una basada en eventos que el programa consulta, y otra basada en callbacks. En este trabajo se utiliza el enfoque por callback, que es el mismo que aparece en el ejemplo de Espressif.

En este enfoque, se registra una función propia del programa que será ejecutada automáticamente cada vez que ocurra un evento táctil. Es decir, cuando el usuario presiona o suelta un botón, la librería llama directamente a esa función, sin necesidad de que el programa esté preguntando constantemente.

Para configurar esto, primero se utiliza touch\_button\_set\_dispatch\_method(), donde se selecciona el modo TOUCH\_ELEM\_DISP\_CALLBACK, indicando que se va a trabajar con callbacks. Luego, mediante la función touch\_button\_set\_callback(), se asocia una función que será ejecutada automáticamente cada vez que ocurra un evento en el botón (Figura 10). Esta función, denominada touch\_button\_callback, actúa como un manejador de eventos y permite procesar la interacción del usuario sin necesidad de realizar consultas constantes desde el programa principal (Espressif Systems, s. f.).

```
ESP_ERROR_CHECK(touch_button_set_dispatch_method(
    button_handle[i],
    TOUCH_ELEM_DISP_CALLBACK
));

ESP_ERROR_CHECK(touch_button_set_callback(|
    button_handle[i],
    touch_button_callback
));
```

*Figura 10: Configuración del método de notificación por callback.*

Cuando la librería llama a esta función le pasa tres parámetros: el identificador del botón (handle), una estructura con información del evento (message) y un puntero genérico (arg) previamente definido al momento de suscribirse a los eventos.

El parámetro message contiene el tipo de evento detectado, como por ejemplo una pulsación (TOUCH\_BUTTON\_EVT\_ON\_PRESS) o una liberación

(TOUCH\_BUTTON\_EVT\_ON\_RELEASE). A partir de este valor, la función puede determinar qué acción realizó el usuario.

Por otro lado, el parámetro `arg` se utiliza para identificar qué botón generó el evento. En este caso, se emplea para pasar el índice del botón, el cual se recupera dentro de la función mediante una conversión de tipo. De esta manera, se puede asociar cada evento con su correspondiente posición en el arreglo de estados.

Dentro de la función `touch_button_callback`, se actualiza un arreglo interno `button_state[]`, donde cada posición representa el estado de un botón. Si el evento corresponde a una pulsación, se asigna el valor `true`, y si corresponde a una liberación, se asigna `false`. De esta forma, se mantiene actualizado el estado de todos los botones en todo momento (Figura 11).

Este enfoque permite desacoplar la detección de eventos del resto del programa. La librería se encarga de detectar las interacciones táctiles y ejecutar la función `callback`, mientras que el programa principal simplemente consulta el estado de los botones a través de este arreglo, facilitando la implementación del polling.

```
static bool button_state[TOUCHPAD_BUTTON_COUNT] = {false};
static bool initialized = false;

static void touch_button_callback(touch_button_handle_t handle,
                                  touch_button_message_t *message,
                                  void *arg)
{
    uint8_t button_index = (uint8_t)(uintptr_t)arg;

    if (message->event == TOUCH_BUTTON_EVT_ON_PRESS) {
        button_state[button_index] = true;
    } else if (message->event == TOUCH_BUTTON_EVT_ON_RELEASE) {
        button_state[button_index] = false;
    }
}
```

*Figura 11: Implementación de la función callback.*

Posteriormente, una vez que la librería se encarga de detectar los eventos y la función `callback` actualiza el estado interno de cada botón, se implementa una función de consulta simple para ser utilizada desde el `main`.

En este caso, se define la función `touchpad_is_pressed(uint8_t button_index)`, cuyo fin es permitir acceder al estado actual de un botón sin necesidad de interactuar directamente con la librería `touch_element` ni con los `callbacks`. Esta función recibe como parámetro el índice del botón que se quiere consultar.

En primer lugar, se verifica que el índice recibido sea válido, es decir, que no exceda la cantidad de botones definidos. En caso contrario, la función devuelve

false, evitando accesos fuera de rango en el arreglo de estados. Por otro lado, Si el índice es válido, la función retorna directamente el valor almacenado en el arreglo `button_state[]` el cual recordemos es actualizado previamente por la función `touch_button_callback`, la cual se ejecuta automáticamente cada vez que ocurre un evento táctil (Figura 12). De esta manera, el valor almacenado en `button_state[]` refleja en todo momento el estado actual de cada botón, siendo true cuando el botón está presionado y false cuando está liberado.

```
bool touchpad_is_pressed(uint8_t button_index)
{
    if (button_index >= TOUCHPAD_BUTTON_COUNT) {
        return false;
    }

    return button_state[button_index];
}
```

*Figura 12: Implementación de la función touchpad\_is\_pressed()*

Finalmente, después de instalar la librería general, instalar el módulo de botones, crear los botones, suscribirse a los eventos y configurar el método de notificación de los mismos, se llama a `touch_element_start()`. Según la documentación, esta función inicia el procesamiento de la librería Touch Element. Recién después de esta llamada el sistema comienza a funcionar y los eventos táctiles pueden ser detectados.

Con la integración de la librería de `delay` y `rgb_led`, el sistema táctil completamente inicializado y el estado de los botones actualizado internamente mediante la función callback, se procede a su utilización desde el programa principal `main.c`.

En primer lugar, se inicializan los módulos necesarios mediante las funciones `rgb_led_init()` y `touchpad_init()`. A continuación, se define un arreglo `last_state[]`, el cual se utiliza para almacenar el estado anterior de cada botón. Este arreglo permite comparar el estado actual con el pasado, lo cual resulta fundamental para detectar cambios de estado, es decir, el instante en que un botón pasa de no estar presionado a estar presionado o viceversa.

Una vez realizadas estas inicializaciones, el programa entra en un bucle infinito `while(1)`, dentro del cual se implementa una lectura por polling. En cada iteración del bucle, se recorren todos los botones mediante un ciclo `for`, y para cada uno se consulta su estado actual utilizando la función `touchpad_is_pressed(i)`.

Para detectar una nueva pulsación, se evalúa la condición `pressed && !last_state[i]`. Esta expresión indica que el botón está actualmente presionado pero que no lo estaba en la iteración anterior, lo que corresponde al

instante en que el usuario realiza la acción de presionar. En ese caso, se imprime un mensaje por consola indicando el botón presionado y se ejecuta una acción asociada al mismo.

En este trabajo, cada botón físico del módulo ESP-LyraP-TouchA se vinculó con una acción específica del LED RGB. En particular, el botón camera enciende el led en violeta, el botón record en amarillo, el botón networken verde, el botón vol+ en rojo, el botón vol- en azul y el botón play/pause apaga completamente el LED.

De manera análoga, se detecta la liberación de un botón mediante la condición `!pressed && last_state[i]`, lo que indica que el botón estaba presionado anteriormente y ahora ya no lo está. En este caso, se imprime un mensaje indicando que el botón fue liberado, aunque no se asocia ninguna acción adicional sobre el LED porque no fue pedido, sino que simplemente se utiliza para mostrar que los botones funcionan como deberían.

Finalmente, al final de cada iteración del ciclo, se actualiza el arreglo `last_state[i]` con el estado actual del botón, de modo que en la siguiente iteración se pueda realizar correctamente la comparación. Se incorpora además un retardo mediante la función `delay_ms()`, lo cual evita que el bucle se ejecute de forma excesivamente rápida y reduce el uso innecesario del procesador (Figura 13).

```

while (1) {
    for (uint8_t i = 0; i < TOUCHPAD_BUTTON_COUNT; i++) {
        bool pressed = touchpad_is_pressed(i);

        if (pressed && !last_state[i]) {
            printf("Boton %d presionado\n", i);

            if (i == 0) {
                rgb_led_set_color(255, 0, 0); // volumen up = rojo
            } else if (i == 1) {
                rgb_led_off(); // play/pause = apaga
            } else if (i == 2) {
                rgb_led_set_color(0, 0, 255); // volumen down = azul
            } else if (i == 3) {
                rgb_led_set_color(255, 255, 0); // record = amarillo
            } else if (i == 4) {
                rgb_led_set_color(255, 0, 255); // camera = violeta (magenta)
            } else if (i == 5) {
                rgb_led_set_color(0, 255, 0); // Network = verde
            }
            //rgb toma (uint8_t red, uint8_t green, uint8_t blue)
        }

        if (!pressed && last_state[i]) {
            printf("Boton %d liberado\n", i);
        }

        last_state[i] = pressed;
    }

    delay_ms(POLLING_PERIOD_MS);
}
}

```

Figura 13: Código principal (*main.c*).

A partir de la implementación realizada, se puede analizar el método utilizado para la detección de botones y compararlo con otras alternativas disponibles en sistemas embebidos. En este trabajo se empleó un enfoque basado en *polling*, el cual consiste en consultar periódicamente el estado de los botones dentro del ciclo principal del programa.

En el enfoque por polling, el microcontrolador ejecuta un bucle en el cual revisa continuamente el estado de cada entrada. Esto implica que la detección de una pulsación depende de la frecuencia con la que se realiza la consulta. En el código implementado, esta verificación se realiza dentro del `while(1)`, recorriendo los botones y utilizando la función `touchpad_is_pressed()` para determinar su estado. Este método es simple de implementar y permite un control directo del flujo del programa, lo cual resulta adecuado para aplicaciones de baja complejidad o donde no se requiere una respuesta inmediata.

Por otro lado, la detección mediante interrupciones se basa en que el hardware genera una señal automáticamente cuando ocurre un evento, interrumpiendo la ejecución normal del programa. En este caso, el microcontrolador no necesita consultar constantemente el estado del botón, sino que responde únicamente cuando ocurre un cambio. Esto permite una mayor eficiencia en el uso del procesador y una respuesta más inmediata ante eventos externos (TutorialsPoint, s. f.).

En términos prácticos, el polling resulta conveniente cuando la frecuencia de lectura es baja, el sistema es simple y no se requiere optimizar el uso del procesador. Además, facilita la implementación y depuración del código, como se observó en este laboratorio. En cambio, el uso de interrupciones es más adecuado en sistemas donde se necesita una respuesta rápida, se manejan múltiples tareas concurrentes o se busca reducir el consumo de CPU evitando consultas constantes.

En este trabajo, si bien la librería `touch_element` utiliza internamente un mecanismo basado en eventos para detectar las interacciones táctiles, el programa principal emplea polling para consultar el estado de los botones. Esto permite combinar ambas estrategias: por un lado, la detección eficiente de eventos a nivel de la librería, y por otro, una interfaz simple de uso desde el `main.c`.

## **2. Conectividad WiFi (AP y STA)**

### **2.1 Infraestructura de red y event loop**

Antes de adentrarse con las configuraciones propias de la conectividad wifi es necesario entender el comportamiento del stack de red de ESP-IDF. Se estudian 2 funciones fundamentales, `esp_netif_init()` y `esp_event_loop_create_default()`. La primera crea estructuras internas y servicios requeridos para las conexiones. Inicializa el TCP/IP, asigna recursos lógicos y garantiza la seguridad de la multitarea, funciones necesarias previas al despliegue de cualquier AP o STA. Si no es inicializada correctamente provoca errores de acceso a memoria y el microcontrolador se reinicia de forma abrupta.

La segunda función `esp_event_loop_create_default()`, es muy relevante para el sistema dado que las operaciones de red son asíncronas (el chip no sabe exactamente en qué milisegundo el router le responderá o le asignará una IP), el programa no puede quedarse "pausado" esperando. Para evitar esto los drivers de más bajo nivel (creando una tarea secundaria) cuando detectan que algo sucede publican un "evento" en el loop y ejecuta el bloque de código asignado al mismo. Esto permite que el ESP32 siga ejecutando otras tareas de la `app_main` o monitoreando el TouchPad mientras el driver de red hace su trabajo en segundo plano.

¿Qué tipos de eventos WiFi se pueden escuchar y para qué sirven?

WIFI\_EVENT\_STA\_CONNECTED se dispara cuando el ESP32 (en modo Estación) logra autenticarse y asociarse físicamente con el router (Access Point objetivo), aunque no es posible enviar datos a internet aún porque todavía no se accede a la dirección ip. Posteriormente se dispara IP\_EVENT\_STA\_GOT\_IP la cual actúa como la "luz verde" que indica que la red está completamente operativa.

Por otro lado, WIFI\_EVENT\_AP\_STACONNECTED se dispara cuando un dispositivo se conecta a la red, dando información útil como la dirección MAC del dispositivo conectado. Esto permite llevar un registro de qué dispositivos se conectan y cuántos están conectados a la red.

## **2.2 Modo Access Point (AP)**

Para generar una red WIFI a la que otros dispositivos pueden conectarse en el entorno ESP-IDF, se toma el ejemplo incluido en la herramienta y se realiza pseudocódigo de este.



```

INICIO app_main:

    // 1. Inicialización Base del Sistema
    INICIALIZAR memoria NVS (necesaria para guardar datos de red)
    INICIALIZAR capa de red (esp_netif_init)
    CREAR event loop por defecto (esp_event_loop_create_default)

    // 2. Registro de Eventos
    REGISTRAR manejador_de_eventos para WIFI_EVENT (conexiones,
desconexiones)

    REGISTRAR manejador_de_eventos para IP_EVENT (obtención de IP)

    // 3. Inicialización del Driver WiFi
    CREAR configuración de wifi por defecto
    (cfg = WIFI_INIT_CONFIG_DEFAULT)
    INICIALIZAR driver wifi con cfg (esp_wifi_init)
    CONFIGURAR modo wifi como AP+STA (WIFI_MODE_APSTA)

    // 4. Configurar Interfaz AP (Tipo router)
    CREAR interfaz netif para AP
    DEFINIR configuracion_ap:
        - SSID, Password, Canal
        - max_connections
        - authmode (WPA2 o OPEN)
    APLICAR configuracion_ap al hardware

    // 5. Configurar Interfaz STA (Conectarse a la red)
    CREAR interfaz netif para STA
    DEFINIR configuracion_sta:
        - SSID objetivo, Password objetivo
    APLICAR configuracion_sta al hardware

    // 6. Arrancar el Hardware
    INICIAR wifi (esp_wifi_start)

    // 7. Esperar y actuar
    MIENTRAS (NO esté conectado a la red STA) Y (NO haya fallado la
conexión):
        ESPERAR 100ms (vTaskDelay) // No bloquear el procesador
    SI (wifi_conectado es VERDADERO):
        IMPRIMIR "¡Conectado al AP externo!"
    CONFIGURAR servidor DNS del AP local para que apunte a la STA
    ESTABLECER interfaz STA como ruta por defecto
        HABILITAR ruteo NAPT (Para compartir internet)
    SINO:
        IMPRIMIR "Error: Fallo al conectar a la red externa"

FIN app_main

```

Figura 14: Pseudocódigo de secuencia de inicialización de AP Y STA

Es importante tener en cuenta que para la realización del código se partió del ejemplo `softap_sta` (Espressif Systems, s. f.). Sin embargo, este contaba con librerías como `free.RTOS`, las cuales todavía no pueden ser utilizadas; por lo tanto, se modificaron partes del código (quitando todo lo relacionado con los `EventGroups`) para evitar su uso reduciendo su complejidad en el proceso.

```
// Usamos esto (volatile es importante para que el compilador sepa que cambia en otro hilo):
static volatile bool wifi_conectado = false;
static volatile bool wifi_fallo = false;

static void wifi_event_handler(void *arg, esp_event_base_t event_base,
                               int32_t event_id, void *event_data)
{
    if (event_base == WIFI_EVENT && event_id == WIFI_EVENT_AP_STACONNECTED) {
        wifi_event_ap_staconnected_t *event = (wifi_event_ap_staconnected_t *) event_data;
        ESP_LOGI(tag: TAG_AP, "Station " MACSTR " joined, AID=%d",
                  MAC2STR(event->mac), event->aid);
    } else if (event_base == WIFI_EVENT && event_id == WIFI_EVENT_AP_STADISCONNECTED) {
        wifi_event_ap_stadisconnected_t *event = (wifi_event_ap_stadisconnected_t *) event_data;
        ESP_LOGI(tag: TAG_AP, "Station " MACSTR " left, AID=%d, reason:%d",
                  MAC2STR(event->mac), event->aid, event->reason);
    } else if (event_base == WIFI_EVENT && event_id == WIFI_EVENT_STA_START) {
        esp_wifi_connect();
        ESP_LOGI(tag: TAG_STA, "Station started");
    } else if (event_base == IP_EVENT && event_id == IP_EVENT_STA_GOT_IP) {
        ip_event_got_ip_t *event = (ip_event_got_ip_t *) event_data;
        ESP_LOGI(tag: TAG_STA, "Got IP: " IPSTR, IP2STR(&event->ip_info.ip));
        s_retry_num = 0;
        wifi_conectado = true; //Señalamos que estamos conectados
    } else if (event_base == IP_EVENT && event_id == IP_EVENT_ASSIGNED_IP_TO_CLIENT) {
        const ip_event_assigned_ip_to_client_t *e = (const ip_event_assigned_ip_to_client_t *) event_data;
        ESP_LOGI(tag: TAG_AP, "Assigned IP to client: " IPSTR ", MAC=" MACSTR ", hostname='%s'",
                  IP2STR(&e->ip), MAC2STR(e->mac), e->hostname);
    }
}
```

Figura 15: Modificaciones ejemplo `softap_sta.c`

```
while(!wifi_conectado && !wifi_fallo) {
    // Es obligatorio usar vTaskDelay para no bloquear el procesador y reinici
    vTaskDelay(xTicksToDelay: 100 / portTICK_PERIOD_MS);
}

if (wifi_conectado) {
    ESP_LOGI(tag: TAG_STA, "¡Conectado al AP!");
    softap_set_dns_addr(esp_netif_ap, esp_netif_sta);
} else {
    ESP_LOGE(tag: TAG_STA, "Fallo al conectar");
}

/* Set sta as the default interface */
esp_netif_set_default_netif(esp_netif: esp_netif_sta);

/* Enable napt on the AP netif */
if (esp_netif_napt_enable(esp_netif: esp_netif_ap) != ESP_OK) {
    ESP_LOGE(tag: TAG_STA, "NAPT not enabled on the netif: %p", esp_netif_ap);
}
```

Figura 16: Modificaciones ejemplo `softap_sta.c` para evitar el uso de eventos.

Las figuras 15 y 16 muestran las modificaciones principales realizadas en el código para su correcto funcionamiento sin el uso de librerías RTOS, sustituyendo la herramienta de eventos mediante una estructura `while`.

Se realizan distintas pruebas con el código modificado para evaluar su correcto funcionamiento en distintas configuraciones.

Configuración inicial:

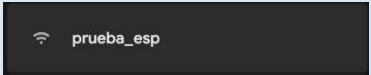
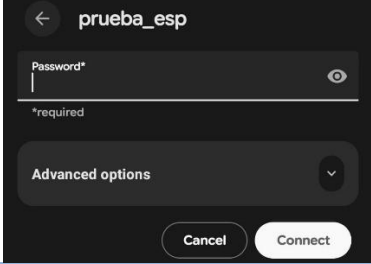
WiFi AP SSID: prueba\_esp  
WiFi AP Password: 12345678  
ESP\_MAX\_STA\_CONN\_AP= 4

```
I (542) WiFi Sta: Station started
I (542) esp_netif_lwip: DHCP server started on interface WIFI_AP_DEF with IP: 192.168.4.1
I (28852) wifi:new:<1,0>, old:<1,1>, ap:<1,1>, sta:<0,0>, prof:1, snd_ch_cfg:0x0
I (28852) wifi:station: 7e:da:4a:72:fe:bc join, AID=1, bgn, 20
I (28982) WiFi SoftAP: Station 7e:da:4a:72:fe:bc joined, AID=1
I (29232) wifi:<ba-add>idx:2 (ifx:1, 7e:da:4a:72:fe:bc), tid:0, ssn:0, winSize:64
I (29462) esp_netif_lwip: DHCP server assigned IP to a client, IP is: 192.168.4.2
I (29462) WiFi SoftAP: Assigned IP to client: 192.168.4.2, MAC=7e:da:4a:72:fe:bc, hostname='Pixel-7-Pro'
I (29552) wifi:<ba-add>idx:3 (ifx:1, 7e:da:4a:72:fe:bc), tid:7, ssn:2, winSize:64
```

Figura 16: Print del monitor al conectar 1 dispositivo al AP.

Cuando un dispositivo se conecta registra el ip asignado al cliente dentro de la red, la direccion MAC y el hostname, en este caso el ip asignado fue 192.168.4.2 el cual es asignado mediante un Servidor DHCP (Protocolo de Configuración Dinámica de Host), el proceso de asignación dinámica de direcciones IP se realiza mediante un intercambio de cuatro mensajes conocidos como DORA (Discover, Offer, Request, Acknowledgment)(GeeksforGeeks, 2023).

Testeo de distintas configuraciones:

| Configuración                | ¿Funciona? | Observaciones                                                                                                                                             |
|------------------------------|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Red abierta (sin contraseña) | Si         | Es necesario borrar el sdkconfig y compilar para ver los cambios.<br> |
| WPA2 con contraseña          | SI         |                                                                       |
| SSID oculto                  | Si         | Al estar oculto se tiene que agregar manualmente la red al dispositivo.                                                                                   |
| Máximo 1 conexión simultánea | Si         | No permite la conexión del segundo dispositivo.                                                                                                           |

2.3 Modo Estación (STA)

Se mostró en la figura 14 la secuencia de inicialización de STA en pseudocódigo. Al realizarse pruebas con el código se agregó un evento para reintentar la conexión a STA e imprimir en el monitor cada intento, teniendo en cuenta el cambio de librerías.

```

} else if (event_base == WIFI_EVENT && event_id == WIFI_EVENT_STA_DISCONNECTED) {
    if (s_retry_num < EXAMPLE_ESP_MAXIMUM_RETRY) {
        esp_wifi_connect();
        s_retry_num++;
        ESP_LOGI(tag: TAG_STA, "Reintentando conectar al AP... (Intento %d)", s_retry_num);
    } else {
        wifi_fallo = true; // Señalamos al bucle while que deje de esperar
        ESP_LOGE(tag: TAG_STA, "Fallo definitivo al conectar al AP");
    }
}

```

Figura 17: Modificación `wifi_event_handler`.

Se configura para realizar la conexión a la siguiente red:

- ESP\_WIFI\_REMOTE\_AP\_SSID: NESSEMO-NET
- SP\_WIFI\_REMOTE\_AP\_PASSWORD: mara8888
- ESP\_WIFI\_SCAN\_AUTH\_MODE\_THRESHOLD: ESP\_WIFI\_AUTH\_WPA\_PSK

```

I (3682) wifi:dp: 1, bi: 102400, li: 3, scale listen interval from 307200 us to 307200 us
I (3742) wifi:AP's beacon interval = 102400 us, DTIM period = 1
I (5792) wifi:<ba-add>idx:0 (ifx:0, 20:08:89:e9:80:f2), tid:0, ssn:2, winSize:64
I (6892) WiFi Sta: Got IP:192.168.1.106
I (6892) esp_netif_handlers: sta ip: 192.168.1.106, mask: 255.255.255.0, gw: 192.168.1.1
I (6922) WiFi Sta: ¡Conectado al AP!

```

Figura 18: Monitor `esp32s2` en modo STA y conectándose a NESSEMO-NET.

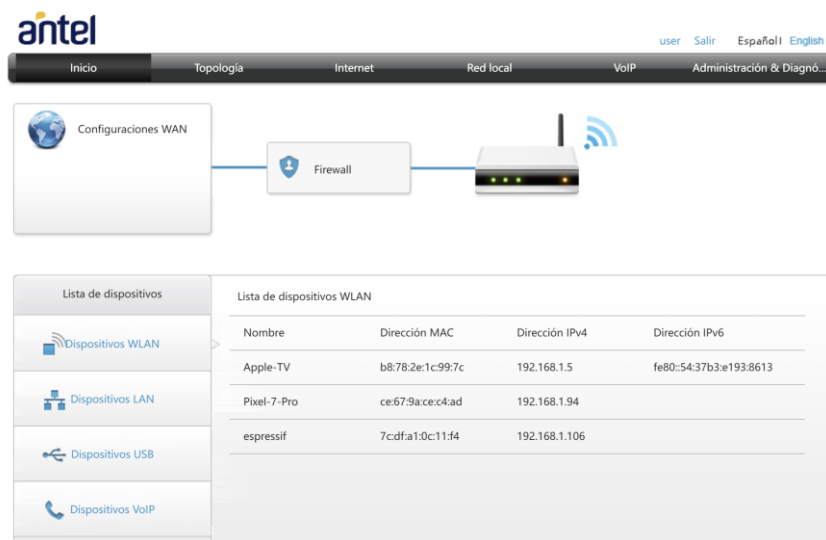


Figura 19: Dispositivos WLAN conectados en la red.

Las figuras 18 y 19 muestran como la conexión a una red con contraseña fue exitosa. Aclarar que en este caso se deshabilitó la funcionalidad de repetidor (`esp_netif_napt_enable(esp_netif_ap)`) para simplificar el funcionamiento del código.

Se configura para realizar la conexión a la siguiente red (OPEN):

- ESP\_WIFI\_REMOTE\_AP\_SSID: Pixel\_7Pro
- SP\_WIFI\_REMOTE\_AP\_PASSWORD: (OPEN)
- ESP\_WIFI\_SCAN\_AUTH\_MODE\_THRESHOLD: ESP\_WIFI\_AUTH\_OPEN

```
I (28972) WiFi Sta: Got IP:10.250.11.13  
I (28972) esp_netif_handlers: sta ip: 10.250.11.13, mask: 255.255.255.0, gw: 10.250.11.102  
I (28972) WiFi Sta: ¡Conectado al AP!
```

*Figura 20: Monitor esp32s2 en modo STA y conectándose a Pixel\_7Pro.*

En vez de asignar un IP como en AP la red a conectarse le brinda una IP mediante el protocolo DHCP, espera a recibir la IP (IP\_EVENT\_STA\_GOT\_IP), imprime los detalles completos de la topología e imprime un mensaje si la conexión fue exitosa en el caso de no serlo reintenta la conexión según la cantidad de intentos configurados.

### 3. Servidor Web embebido

#### 3.1 "Hola Mundo" web

El objetivo de esta primera instancia fue comprobar que el ESP32-S2 pudiera mostrar una página web simple desde la placa. Para ello, la consigna pide incorporar `esp_http_server`, levantar un servidor HTTP y acceder desde un navegador a `http://<ip-del-esp32>/`, donde debía mostrarse el texto “Hola Mundo”.

`esp_http_server` es el componente de ESP-IDF que permite que el ESP32-S2 funcione como un servidor web. Es decir, el microcontrolador queda esperando solicitudes realizadas desde un navegador y responde con texto o contenido web. Según la documentación de Espressif, este componente permite iniciar un servidor HTTP mediante `httpd_start()` y asociar una ruta, por ejemplo `/`, con una función que define qué respuesta se enviará al navegador.

En primer lugar, se modificó el archivo `CMakeLists.txt` del proyecto para agregar `esp_http_server` dentro de las dependencias. Esto fue necesario porque el programa principal utiliza funciones propias del servidor HTTP, como `httpd_start()`, `HTTPD_DEFAULT_CONFIG()`, `httpd_resp_send()` y `httpd_register_uri_handler()`. Sin esta dependencia, el compilador no reconoce esas funciones y el proyecto no puede compilar correctamente.

La secuencia mínima diseñada para levantar el servidor fue la siguiente:

- Inicializar la red y el WiFi como en la segunda parte.
- Configurar el ESP32-S2 en modo AP.
- Iniciar el WiFi.
- Crear la configuración básica del servidor HTTP.
- Iniciar el servidor.
- Indicar que, cuando llegue una solicitud a la ruta `/`, se ejecute una función de respuesta.
- Dentro de esa función, enviar al navegador el texto `"Hola Mundo"`.

Luego se implementó el servidor en el archivo `main.c`. Para esto se creó la función `start_webserver()`, que tiene como finalidad inicializar el servidor HTTP y dejarlo pronto para responder solicitudes realizadas desde un navegador. Dentro de esta función se declara primero una variable de tipo `httpd_handle_t`, llamada `server`, que funciona como una referencia al servidor que se va a crear. A continuación, se define la configuración inicial del servidor

mediante `HTTPD_DEFAULT_CONFIG()` y se llama a `httpd_start(&server, &config)`, que es la función que inicia efectivamente el servidor HTTP. Si esta llamada devuelve `ESP_OK`, significa que el servidor se inició correctamente y que la placa ya puede recibir solicitudes HTTP.

```
//inicialización del servidor
httpd_handle_t start_webserver(void)
{
    httpd_handle_t server = NULL;
    httpd_config_t config = HTTPD_DEFAULT_CONFIG();

    // Iniciar el servidor
    if (httpd_start(&server, &config) == ESP_OK) {
        // Registrar URI handler
        httpd_uri_t uri_get = {
            .uri      = "/",
            .method    = HTTP_GET,
            .handler   = http_get_handler,
            .user_ctx  = NULL
        };
        httpd_register_uri_handler(server, &uri_get);
    }
    return server;
}
```

*Figura 21: Inicialización del servidor HTTP.*

Una vez iniciado el servidor, se define la ruta principal "/" mediante una estructura de tipo `httpd_uri_t`. En dicha estructura se indica que la ruta corresponde al método `HTTP_GET`, que es la forma en que el navegador solicita una página al servidor para poder visualizarla. También se indica que, cuando llegue una solicitud a esa ruta, debe ejecutarse la función `http_get_handler()` (Figura 21).

La función `http_get_handler()` es la encargada de generar la respuesta que recibe el navegador. En esta primera prueba, la función manda un texto simple mediante `httpd_resp_send()`. El texto enviado fue "Hola Mundo" (Figura 22), por lo que, al ingresar desde el navegador a la dirección del ESP32-S2, la página mostraba únicamente ese mensaje. De esta forma se comprobó que el servidor estaba funcionando correctamente y que el navegador podía recibir una respuesta generada desde la placa.

```
//handler
esp_err_t http_get_handler(httpd_req_t *req)
{
    const char* resp_str = "Hello, World!";
    httpd_resp_send(req, resp_str, HTTPD_RESP_USE_STRLEN);

    return ESP_OK;
}
```

*Figura 22: Respuesta "Hello, World!" desde el servidor.*

La dirección utilizada para acceder al servidor fue 192.168.4.1, correspondiente a la IP del ESP32-S2 en modo AP, como se había visto en la segunda parte del laboratorio.

### 3.2 Página web embebida

Luego de comprobar que el servidor podía responder con un texto simple, se avanzó hacia una forma más ordenada de mostrar contenido web. En lugar de escribir el contenido de la página directamente dentro de `main.c`, se creó una carpeta llamada `web` dentro de `main/`, donde se ubicaron los archivos propios de la página. El archivo principal es `index.html`, ya que allí se define la estructura de lo que se verá en el navegador, por ejemplo los textos, títulos, botones, entradas o secciones de la página. Además, la consigna plantea la posibilidad de agregar `style.css`, utilizado para definir la apariencia visual de la página, y `app.js`, que podría utilizarse para separar el código JavaScript encargado de incorporar acciones desde el navegador, como responder a botones o enviar datos al ESP32-S2. En este caso, la lógica necesaria para la interacción con la página fue incluida directamente dentro del archivo `index.html`.

En el proyecto se agregaron los archivos `web/index.html`, `web/style.css` y `web/app.js` mediante la opción `MBED_FILES` del archivo `CMakeLists.txt`. Esta opción permite que dichos archivos queden incluidos dentro del programa que se carga en el ESP32-S2. De esta forma, la placa no necesita buscar el archivo HTML en una memoria externa ni en una computadora, sino que el servidor web puede tomarlo directamente desde el firmware y enviarlo al navegador cuando se accede a la página.

Para poder enviar el archivo `index.html` desde `main.c`, se declararon dos referencias externas: `index_html_start` e `index_html_end`. Estas referencias son generadas automáticamente por ESP-IDF cuando el archivo se agrega en `MBED_FILES`. La primera indica desde qué posición de memoria comienza el contenido de `index.html`, y la segunda indica dónde termina. Con esas dos posiciones, el programa puede calcular el tamaño total del archivo y enviarlo completo al navegador mediante el servidor HTTP.

```

/* HTML de index embebido */
extern const uint8_t index_html_start[] asm("_binary_index_html_start");
extern const uint8_t index_html_end[]   asm("_binary_index_html_end");

```

Figura 23: Referencias externas al archivo index.html.

```

/* Respuesta para cargar la página principal */
esp_err_t http_get_handler(httpd_req_t *req)
{
    const char *html = (const char *) index_html_start;
    size_t html_len = index_html_end - index_html_start;

    httpd_resp_set_type(req, "text/html");
    httpd_resp_send(req, html, html_len);

    return ESP_OK;
}

```

Figura 24: Envío del archivo index.html al navegador.

La primera línea toma la dirección donde empieza el archivo index.html (Figura 23). Luego se calcula su largo restando la dirección final menos la dirección inicial. Después, mediante `httpd_resp_set_type(req, "text/html")`, se le indica al navegador que la respuesta recibida debe interpretarse como una página HTML. Finalmente, `httpd_resp_send(req, html, html_len)` envía el contenido completo de esa página (Figura 24).

En el archivo index.html se empezó a escribir la estructura de la página que se muestra en el navegador.

HTML funciona mediante etiquetas, que se escriben entre los signos `< >`. Cada etiqueta indica qué tipo de elemento se quiere mostrar. Por ejemplo, `<h1>` se usa para un título principal, `<br>` introduce saltos de línea, `<p>` para un párrafo y `<button>` para un botón. Generalmente las etiquetas se abren y cierran, como `<p>Texto</p>`, donde el contenido queda entre ambas etiquetas.

En este caso, el archivo comienza indicando que se trata de un documento HTML mediante `<!DOCTYPE html>`. Luego se utiliza la etiqueta `<html>` para marcar el inicio de la página y la etiqueta `<body>` para contener todo lo que se verá desde el navegador (W3Schools, s. f.). Dentro de esa sección se agregó primero el título Hello world, probando, utilizando la etiqueta `<h1>`. Este texto luego fue cambiado a "EAMTA".

Después se agregó una segunda sección para mandar colores al led. Para ingresar los valores de color se agregaron tres entradas numéricas donde cada



una corresponde a un componente del color RGB: rojo, verde y azul. En cada input se colocó `type="number"` lo que indica que el usuario debe ingresar un número. Por otro lado se especifica, `min="0"` y `max="255"` que limitan el rango permitido, porque cada componente RGB puede tomar valores entre 0 y 255. Además, cada entrada tiene un identificador distinto: `r`, `g` y `b`. Estos identificadores son importantes porque después JavaScript los usa para leer los valores ingresados(Figura 25).

```
<h2> Cambiar color de led <br></h2>

R: <input type="number" id="r" name="r" min="0" max="255"><br>
G: <input type="number" id="g" name="g" min="0" max="255"><br>
B: <input type="number" id="b" name="b" min="0" max="255"><br>
```

Figura 25: Entradas RGB en la página web.

Posteriormente, se agregó el botón `Cambiar color`, que permite enviar los valores ingresados por el usuario cuando se lo presiona. Este botón ejecuta la función `Mandar_Color()`(Figura 26), definida más abajo dentro de la etiqueta `<script>` y explicada con mayor detalle en la siguiente sección. Dicha función toma los valores escritos en las entradas `R`, `G` y `B`, y los envía al ESP32-S2.

```
<button type="button" onclick='Mandar_Color()> Cambiar color </button>
```

Figura 26: Botón para enviar el color al ESP32-S2.

### 3.3 Control del LED RGB desde el navegador

La función `Mandar_Color()` empieza leyendo los valores ingresados en los campos de la página. Para eso utiliza `document.getElementById("r").value`, `document.getElementById("g").value` y `document.getElementById("b").value`. Cada una de estas instrucciones busca el campo correspondiente según su identificador y obtiene el valor escrito por el usuario. Luego, esos valores se pasan a números enteros con `parseInt()` porque los datos leídos desde una entrada HTML llegan inicialmente como texto.

Para enviar los valores a la placa se usó JSON. JSON significa *JavaScript Object Notation* y es un formato para intercambiar datos. Permite representar información mediante pares de nombre y valor, por ejemplo `{ "r": 255, "g": 0, "b": 128 }`. En este caso, los nombres `r`, `g` y `b` identifican cada componente del color, y los valores numéricos indican la intensidad de rojo, verde y azul que debe aplicarse al LED (JSON.org ,s. f.).

Además de JSON, existen otras formas de serializar datos en sistemas embebidos. Una opción simple sería enviar los valores como texto plano, por ejemplo `255,0,128`, y luego separar esos valores en C. También podrían enviarse como parámetros en la URL, por ejemplo `/led?r=255&g=0&b=128`, ya que existen herramientas como `URLSearchParams` para trabajar con este tipo de

parámetros. Otra alternativa es usar formatos binarios más compactos, como CBOR o Protocol Buffers. CBOR está pensado para permitir mensajes de tamaño reducido, mientras que Protocol Buffers permite serializar datos estructurados de forma más compacta que otros formatos textuales (MDN\_, 2023; Bormann & Hoffman, 2020; Protocol Buffers, s. f.).

En el código en JavaScript (<script>), el mensaje se arma con `JSON.stringify({ r: parseInt(r), g: parseInt(g), b: parseInt(b) })`. Esta línea de código toma los valores leídos desde la página y los convierte en un texto con formato JSON. Por ejemplo, si el usuario ingresa 255, 255 y 255, el mensaje enviado al ESP32-S2 tiene la forma `{ "r": 255, "g": 255, "b": 255 }`.

Luego se utiliza `fetch()` para enviar esa información al servidor web del ESP32-S2. En este caso, la solicitud se envía a la dirección `/led` y usa el método POST, porque el navegador está mandando datos para modificar el LED. Esta separación permite que `/` se use solamente para cargar la página principal `index.html`, mientras que `/led` queda destinada al intercambio de información relacionada con el LED. Además, se agrega el encabezado `Content-Type: application/json`, que indica que los datos enviados tienen formato JSON.

Finalmente, se agregaron respuestas para saber si el envío ocurrió correctamente. La función `fetch()` espera una respuesta del servidor. Si la placa recibe bien los datos y responde, se ejecuta la parte del código asociada a `.then()`. Ahí primero se lee la respuesta como texto mediante `response.text()`, después esa respuesta se imprime en la consola del navegador con `console.log()`, y también se muestra una ventana emergente con el mensaje "Color cambiado!".

Después de recibir la confirmación del servidor, se llama a la función `Consultar_Estado_LED()`. Esta llamada permite actualizar en la página el estado mostrado del LED inmediatamente después de cambiar el color, sin tener que recargar manualmente la página.

En cambio, si ocurre algún problema durante el envío de los datos, por ejemplo si el servidor no responde o se pierde la conexión con el ESP32-S2, se ejecuta la parte de `.catch()`. En ese caso, el error se imprime en la consola con `console.error()` y se muestra una alerta indicando Error al cambiar el color (Figura 27).

```

<script>
  function Mandar_Color() {
    const r = document.getElementById("r").value;
    const g = document.getElementById("g").value;
    const b = document.getElementById("b").value;

    fetch('/', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({ r: parseInt(r), g: parseInt(g), b: parseInt(b) })
    })
    .then(response => response.text())
    .then(data => {
      console.log('Respuesta:', data);
      alert('Color cambiado!');
    })
    .catch((error) => {
      console.error('Error:', error);
      alert('Error al cambiar el color');
    });
  }
</script>

```

*Figura 27: Envío de valores RGB mediante fetch().*

Del lado del ESP32-S2, el servidor recibe esa solicitud en la función `http_led_post_handler()`. Esta función es la encargada de procesar los datos que llegan desde el navegador y convertirlos en un cambio visible en el LED. En primer lugar, se obtiene el tamaño del mensaje recibido mediante `req->content_len`, ya que el cuerpo de la solicitud HTTP contiene el JSON enviado desde la página. Luego se reserva memoria para guardar ese contenido.

Después, el contenido de la solicitud se lee mediante `httpd_req_recv()`. Como el mensaje puede no llegar completo en una sola lectura, el código utiliza un ciclo `while` que sigue leyendo hasta recibir todos los bytes indicados por `total_len`. Si ocurre un error durante la recepción, se libera la memoria reservada con `free(buf)` y se responde al navegador con un error mediante `httpd_resp_send_err()`. Esto evita que el programa continúe trabajando con datos incompletos o inválidos.

Una vez recibido todo el mensaje, se agrega `buf[total_len]='\0'`. Esto asegura que el texto recibido termine correctamente como string en C. A partir de ese momento, el contenido puede ser interpretado como un mensaje JSON.

Luego se utiliza `cJSON_Parse(buf)` para interpretar ese texto. `cJSON` es una librería escrita en C que permite trabajar con datos en formato JSON. En lugar de tratar el mensaje recibido como un string común, `cJSON` lo interpreta y lo

convierte en una estructura interna que puede consultarse por campos (lo parsea). Por eso, a partir del mensaje { "r": 255, "g": 0, "b": 128 }, el programa puede obtener directamente los campos r, g y b, sin tener que recorrer manualmente todo el texto recibido para distinguir llaves, comillas, nombres de campos, separadores y valores numéricos. Esto reduce la posibilidad de errores al interpretar el mensaje y permite controlar de forma más clara casos en los que falte un campo o el formato recibido no sea válido (Espressif Systems, 2026).

Si `cJSON_Parse()` falla, significa que el contenido recibido no tiene un formato JSON válido. En ese caso, el programa libera la memoria reservada, responde con el error JSON inválido y termina la función. Si el JSON es válido, se buscan los campos necesarios mediante `cJSON_GetObjectItem(json, "r")`, `cJSON_GetObjectItem(json, "g")` y `cJSON_GetObjectItem(json, "b")`. Estas funciones permiten obtener por separado los valores asociados a cada componente del color. Antes de usar esos valores, el programa verifica que los tres campos sean numéricos mediante `cJSON_IsNumber()`. Esta verificación es importante porque evita aplicar valores incorrectos al LED. Si alguno de los campos falta o no es un número, se responde al navegador con un error indicando que faltan campos o que no son válidos. En cambio, si los tres valores son correctos, se llama a `set_led_state(r->valueint, g->valueint, b->valueint)`.

La función `set_led_state()` se agregó para que el cambio del LED y el guardado de su estado se realicen en un mismo lugar. Esta función recibe los valores r, g y b, los guarda en variables llamadas `led_r`, `led_g` y `led_b`, y luego determina si el LED debe considerarse encendido o apagado. Para esto se usa la variable `led_on`, si r, g y b valen cero, se considera que el LED está apagado; si al menos uno de esos valores es distinto de cero, se considera que está encendido. Finalmente, la función llama a `rgb_led_set_color(r, g, b)`, que es la función de la librería `rgb_led` encargada de aplicar físicamente el color al LED RGB de la placa.

Una vez actualizado el LED, el servidor responde con el mensaje LED actualizado mediante `httpd_resp_sendstr()`. Esta respuesta es la que luego recibe la función `fetch()` en la página web. Finalmente, el código libera la memoria utilizada por el JSON con `cJSON_Delete(json)` y libera también el buffer reservado con `free(buf)`. Esto es necesario porque tanto el mensaje recibido como la estructura creada por `cJSON` ocupan memoria dinámica durante la ejecución.

```

/* Recibe JSON por POST /led y actualiza el LED */
esp_err_t http_led_post_handler(httpd_req_t *req)
{
    int total_len = req->content_len;
    char *buf = malloc(total_len + 1);
    int recibido = 0;

    if (!buf) {
        httpd_resp_send_err(req, HTTPD_500_INTERNAL_SERVER_ERROR, "No hay memoria");
        return ESP_FAIL;
    }

    while (recibido < total_len) {
        int len = httpd_req_recv(req, buf + recibido, total_len - recibido);

        if (len <= 0) {
            free(buf);
            httpd_resp_send_err(req, HTTPD_400_BAD_REQUEST, "Error al recibir datos");
            return ESP_FAIL;
        }

        recibido += len;
    }

    buf[total_len] = '\0';

    cJSON *json = cJSON_Parse(buf);

    if (!json) {
        free(buf);
        httpd_resp_send_err(req, HTTPD_400_BAD_REQUEST, "JSON inválido");
        return ESP_FAIL;
    }
}

```

Figura 28: Recepción del JSON en POST /led.

```

cJSON *r = cJSON_GetObjectItem(json, "r");
cJSON *g = cJSON_GetObjectItem(json, "g");
cJSON *b = cJSON_GetObjectItem(json, "b");

if (cJSON_IsNumber(r) && cJSON_IsNumber(g) && cJSON_IsNumber(b)) {
    set_led_state(r->valueint, g->valueint, b->valueint);
} else {
    cJSON_Delete(json);
    free(buf);
    httpd_resp_send_err(req,
                        HTTPD_400_BAD_REQUEST,
                        "Faltan campos r, g o b o no son números");
    return ESP_FAIL;
}

httpd_resp_sendstr(req, "LED actualizado");

cJSON_Delete(json);
free(buf);

return ESP_OK;

```

*Figura 29: Validación y aplicación del color al LED.*

Además del cambio de color mediante POST /led, se agregó una forma de consultar el estado actual del LED. Para esto se implementó la función `http_led_get_handler()` (Figura 28), asociada a la misma ruta /led, pero usando el método GET. Esta función no cambia el color del LED, sino que responde con los últimos valores guardados en `led_r`, `led_g`, `led_b` y `led_on`.

Para enviar esa información al navegador, `http_led_get_handler()` arma un mensaje en formato JSON utilizando `snprintf()`, que permite formar una cadena con los valores actuales del LED y guardarla temporalmente en un arreglo de caracteres. Luego se indica que la respuesta es de tipo `application/json` mediante `httpd_resp_set_type()`, y finalmente se envía al navegador con `httpd_resp_sendstr()` (Figura 29).

En la función `start_webserver()` también se modificó el registro de rutas del servidor. La ruta / con método GET se mantiene para cargar la página principal `index.html`. Por otro lado, se agregó la ruta /led que se utiliza para las operaciones relacionadas con el LED. Cuando llega una solicitud POST /led, se ejecuta `http_led_post_handler()` y se cambia el color del LED. Cuando llega una solicitud GET /led, se ejecuta `http_led_get_handler()` y se devuelve el estado actual del LED en formato JSON.

Del lado de la página web, también se agregaron elementos para mostrar ese estado. En `index.html` se incorporaron textos identificados con

id="estado\_led" e id="color\_led". El primero se utiliza para mostrar si el LED está encendido o apagado, y el segundo para mostrar los valores actuales de color.

Para actualizar esos textos se agregó la función `Consultar_Estado_LED()`. Esta función realiza una solicitud GET a `/led`, es decir, le pide al ESP32-S2 el estado actual del LED. Como el servidor responde en formato JSON, la página interpreta esa respuesta y usa sus campos para modificar lo que se muestra en el navegador. Si "on" vale true, se muestra que el LED está prendido; si vale false, se muestra que está apagado. Además, se actualizan los valores de R, G y B en la página.

También se modificó la etiqueta `<body>` agregando `onload="Consultar_Estado_LED()"`. Esto significa que, cuando la página termina de cargarse, se ejecuta automáticamente la función `Consultar_Estado_LED()`. De esta manera, al abrir o recargar la página, el navegador consulta a la placa cuál es el estado real del LED y lo muestra en pantalla.

Por último, después de cambiar el color con el botón Cambiar color, la función `Mandar_Color()` vuelve a llamar a `Consultar_Estado_LED()`. Esto permite que la página se actualice inmediatamente después de recibir la confirmación del servidor, sin necesidad de recargarla manualmente.

### 3.4 Integración: TouchPad + LED + Web

Luego de implementar el control del LED RGB desde el navegador, se modificó la interfaz para que la selección del color fuera más práctica. En lugar de ingresar manualmente los valores de rojo, verde y azul mediante campos numéricos, se agregaron tres sliders, uno para cada componente RGB. Estos controles se implementaron con entradas de tipo range, con valores entre 0 y 255, que es el rango utilizado para representar cada componente de color (Figura 30).

```
R: <input type="range" id="r" min="0" max="255" value="0">
<span id="rVal">0</span><br>

G: <input type="range" id="g" min="0" max="255" value="0">
<span id="gVal">0</span><br>

B: <input type="range" id="b" min="0" max="255" value="0">
<span id="bVal">0</span><br>
```

*Figura 30: Implementación de sliders.*

Cada slider se asoció a un identificador propio: r, g y b. Además, junto a cada uno se agregó un elemento span, identificado como rVal, gVal y bVal, utilizado para mostrar en la página el valor numérico actual de cada componente. De esta manera, cuando el usuario mueve un slider, no solo cambia internamente el valor que será enviado al ESP32-S2, sino que también se actualiza inmediatamente el número mostrado en pantalla.

Para detectar el movimiento de los sliders se utilizó el evento `oninput`. En el código se asignó la misma función a los tres controles, por lo que cualquier modificación en rojo, verde o azul ejecuta el mismo bloque de instrucciones. Dentro de este bloque, primero se copian los valores actuales de los sliders hacia los elementos `span`, utilizando `textContent`. También se agregó una variable `timeout` (del tipo `let` porque va cambiando), utilizada para controlar el envío de datos mientras el usuario mueve los sliders. Cada vez que ocurre un cambio, se cancela el envío anterior mediante `clearTimeout(timeout)` y se programa uno nuevo con `setTimeout()`. Esto evita que se envíen demasiadas solicitudes seguidas al servidor mientras el usuario está moviendo el slider. Recién después de ese pequeño retardo se envían los valores actuales al ESP32-S2.

```
r.oninput = g.oninput = b.oninput = () => {  
  
  // Actualizar números mostrados  
  rVal.textContent = r.value;  
  gVal.textContent = g.value;  
  bVal.textContent = b.value;  
  
  // Cancelar envío anterior  
  clearTimeout(timeout);  
  
  // Esperar un poco antes de enviar  
  timeout = setTimeout(() => {  
  
    fetch('/led', {  
      method: 'POST',  
      headers: {  
        'Content-Type': 'application/json'  
      },  
      body: JSON.stringify({  
        r: parseInt(r.value),  
        g: parseInt(g.value),  
        b: parseInt(b.value)  
      })  
    })  
  
    .then(response => response.text())  
    .then(data => {  
  
      console.log('Respuesta:', data);  
  
      document.getElementById("estado_led").innerHTML =  
        "Estado: encendido";  
  
      document.getElementById("color_led").innerHTML =  
        "Color actual: R=" + r.value +  
        ", G=" + g.value +  
        ", B=" + b.value;  
    })  
    .catch(error => {  
      console.error('Error:', error);  
    })  
    .then(() => {  
      // Esperar 10 segundos antes de volver a enviar  
      setTimeout(() => {  
        fetch('/led', {  
          method: 'POST',  
          headers: {  
            'Content-Type': 'application/json'  
          },  
          body: JSON.stringify({  
            r: parseInt(r.value),  
            g: parseInt(g.value),  
            b: parseInt(b.value)  
          })  
        })  
      })  
    })  
  }, 10);  
});
```

*Figuras 31 y 32: Manejo de los inputs de los sliders.*

Después de incorporar los sliders, también se modificó la forma en que la página muestra los valores actuales. Cada vez que se mueve un slider, se actualiza el número que tiene asociado, es decir, el valor mostrado junto a cada componente RGB. Además, cuando la página se carga o se recarga, se ejecuta la función `syncColorFromServer()`. Esta función consulta al ESP32-S2 cuál es el estado actual del LED y, con esa información, actualiza los textos de estado y color. Ahora también modifica directamente la posición de los sliders mediante `r.value`, `g.value` y `b.value`, y actualiza los números asociados a cada uno mediante `rVal`, `gVal` y `bVal`. Por eso, al abrir la página, los sliders no quedan siempre en cero, sino que se acomodan al último color guardado en la placa.



```

function syncColorFromServer() {
    fetch('/led')
        .then(response => response.json())
        .then(data => {
            r.value = data.r;
            g.value = data.g;
            b.value = data.b;

            rVal.textContent = data.r;
            gVal.textContent = data.g;
            bVal.textContent = data.b;

            document.getElementById("estado_led").innerHTML =
                "Estado: " + (data.r == 0 && data.g == 0 && data.b == 0 ? "apagado" : "encendido");

            document.getElementById("color_led").innerHTML =
                "Color actual: R=" + data.r +
                ", G=" + data.g +
                ", B=" + data.b;
        })
        .catch(error => {
            console.error('Error al obtener estado del LED:', error);
        });
}

// Consultar el estado del LED cada 500 milisegundos (0.5 segundos)
setInterval(syncColorFromServer, 500);

// Ejecutarlo una vez al abrir la página para obtener el color inicial
syncColorFromServer();

```

Figura 33: Control del estado actual del led.

A partir de esto se integró el TouchPad al proyecto. Para hacerlo se reutilizó la misma idea implementada en la sección 1: el programa recorre los botones dentro de un while sin fin, consulta el estado de cada uno con `touchpad_is_pressed(i)` y compara ese valor con el estado anterior guardado en `last_state[i]`. De esta forma se detecta solamente el instante en que un botón pasa de no estar presionado a estar presionado. La diferencia respecto a la primera parte es que, en lugar de llamar directamente a la función de la librería `rgb_led`, ahora cada botón llama a `set_led_state()`, función explicada en la sección anterior. Esto permite que, además de cambiar físicamente el color del LED, también se actualicen las variables internas que guardan el estado actual del sistema. Así, cuando la página consulta el estado mediante GET `/led`, recibe el último color seleccionado, aunque ese cambio haya sido realizado desde el TouchPad.

```

while (1) {
    for (uint8_t i = 0; i < TOUCHPAD_BUTTON_COUNT; i++) {
        bool pressed = touchpad_is_pressed(i);

        if (pressed && !last_state[i]) {
            if (i == 0) {
                set_led_state(255, 0, 0); // rojo
            } else if (i == 1) {
                set_led_state(0, 0, 0); // apagado
            } else if (i == 2) {
                set_led_state(0, 0, 255); // azul
            } else if (i == 3) {
                set_led_state(255, 255, 0); // amarillo
            } else if (i == 4) {
                set_led_state(255, 0, 255); // violeta
            } else if (i == 5) {
                set_led_state(0, 255, 0); // verde
            }
        }

        last_state[i] = pressed;
    }

    delay_ms(50);
}

```

Figura 34: Control del estado actual del led.

## Referencias

- Bormann, C., & Hoffman, P. (2020). *RFC 8949: Concise Binary Object Representation (CBOR)*. RFC Editor. Consultado el 12 de mayo de 2026 de <https://www.rfc-editor.org/rfc/rfc8949.html>
- Espressif Systems. (s. f.). *Capacitive touch sensor - ESP32-S2 - ESP-IDF programming guide*. Consultado el 24 de abril de 2026 de [https://docs.espressif.com/projects/esp-idf/en/stable/esp32s2/api-reference/peripherals/cap\\_touch\\_sens.html](https://docs.espressif.com/projects/esp-idf/en/stable/esp32s2/api-reference/peripherals/cap_touch_sens.html)
- Espressif Systems. (2026). *cJSON*. ESP Registry. Consultado el 12 de Mayo de 2026 de <https://components.espressif.com/components/espressif/cjson>
- Espressif Systems. (s. f.). *Touch element*. ESP-IDF Extra Components Documentation. Consultado el 25 de abril de 2026 de [https://espressif.github.io/idf-extra-components/latest/touch\\_element/index.html](https://espressif.github.io/idf-extra-components/latest/touch_element/index.html)
- Espressif Systems. (s. f.). *Touch sensor - ESP32-S2 - ESP-IDF programming guide*. Consultado el 24 de abril de 2026 de [https://docs.espressif.com/projects/esp-idf/en/v4.4/esp32s2/api-reference/peripherals/touch\\_pad.html](https://docs.espressif.com/projects/esp-idf/en/v4.4/esp32s2/api-reference/peripherals/touch_pad.html)
- Espressif Systems. (s. f.). WiFi softAP & station Example (softap\_sta.c). Consultado el 25 de abril de 2026 de [https://github.com/espressif/esp-idf/blob/master/examples/wifi/softap\\_sta/main/softap\\_sta.c](https://github.com/espressif/esp-idf/blob/master/examples/wifi/softap_sta/main/softap_sta.c)
- Espressif Systems. (s.f). *ESP32-S2-Kaluga-1 kit v1.2 user guide*. Consultado el 24 de abril de 2026 de <https://docs.espressif.com/projects/esp-idf/en/v4.3/esp32s2/hw-reference/esp32s2/user-guide-esp32-s2-kaluga-1-kit-v1.2.html>
- Espressif Systems. (s.f). *ESP-LyraP-TouchA v1.1 user guide*. Consultado el 24 de abril de 2026 de <https://docs.espressif.com/projects/esp-idf/en/v4.3/esp32s2/hw-reference/esp32s2/user-guide-esp-lyrap-toucha-v1.1.html>
- GeeksforGeeks. (2023). *How DORA Works?* Recuperado el 27 de abril de 2026 de <https://www.geeksforgeeks.org/computer-networks/how-dora-works/>

JSON.org. (s. f.). *Introducing JSON*. Consultado el 12 de mayo de 2026 de <https://www.json.org/>

TutorialsPoint. (s. f.). *Difference between interrupt and polling in OS*. Consultado el 25 de abril de 2026 de <https://www.tutorialspoint.com/article/difference-between-interrupt-and-polling-in-os>

MDN\_. (2023). *URLSearchParams*. Consultado el 12 de mayo de 2026 de <https://developer.mozilla.org/es/docs/Web/API/URLSearchParams>

Protocol Buffers Documentation. (s. f.). *Protocol Buffers*. Consultado el 12 de mayo de 2026 de <https://protobuf.dev/overview/>

W3Schools. (s. f.). HTML basic examples. Consultado el 12 de mayo de 2026 de [https://www.w3schools.com/html/html\\_basic.asp](https://www.w3schools.com/html/html_basic.asp)